

ALU Connect

A Role-Based Mobile Platform Connecting ALU Students with Campus Startup Opportunities

Technical Report — Cedrick Bienvenue · 12 July 2026

GITHUB REPOSITORY

github.com/cedrick13bienvenue/internshipConnector

DEMO VIDEO

<https://youtu.be/oFa65CZeCs4>

Abstract

ALU Connect is a Flutter-based mobile application that bridges ALU students seeking internship experience with student-led startups and early-stage ventures on campus. The platform allows verified startups to post opportunities and enables students to discover, apply for, and track applications in real time. This report documents the system architecture, Firebase backend structure, state management approach, application workflows, UI/UX reasoning, scalability considerations, engineering challenges, testing strategy, limitations, and future improvements for the platform, built with Flutter 3.32, Dart 3.8, Firebase Authentication, Cloud Firestore, and the BLoC/Cubit state management pattern.

1. Introduction & Problem Statement

Many ALU students struggle to secure internship experience at established organizations, while student entrepreneurs running startups within the ALU ecosystem often need support in software development, design, marketing, operations, research, business analysis, content creation, and community management — yet have no reliable channel to reach student talent. ALU Connect addresses this gap directly: it gives startups a way to post real opportunities and gives students a single place to discover, filter, and apply to them, replacing the informal word-of-mouth and scattered group-chat postings that currently mediate this exchange.

The platform's core design goal was not to build a generic job board, but to reflect the specific dynamics of the ALU startup ecosystem: startups need a lightweight but credible verification step before they can post, applications need a visible pipeline so students aren't left wondering about status, and both sides need real-time feedback when something changes.

2. System Architecture

ALU Connect follows a layered architecture that separates presentation, state, and data-access concerns, keeping the codebase testable and maintainable as features grow.

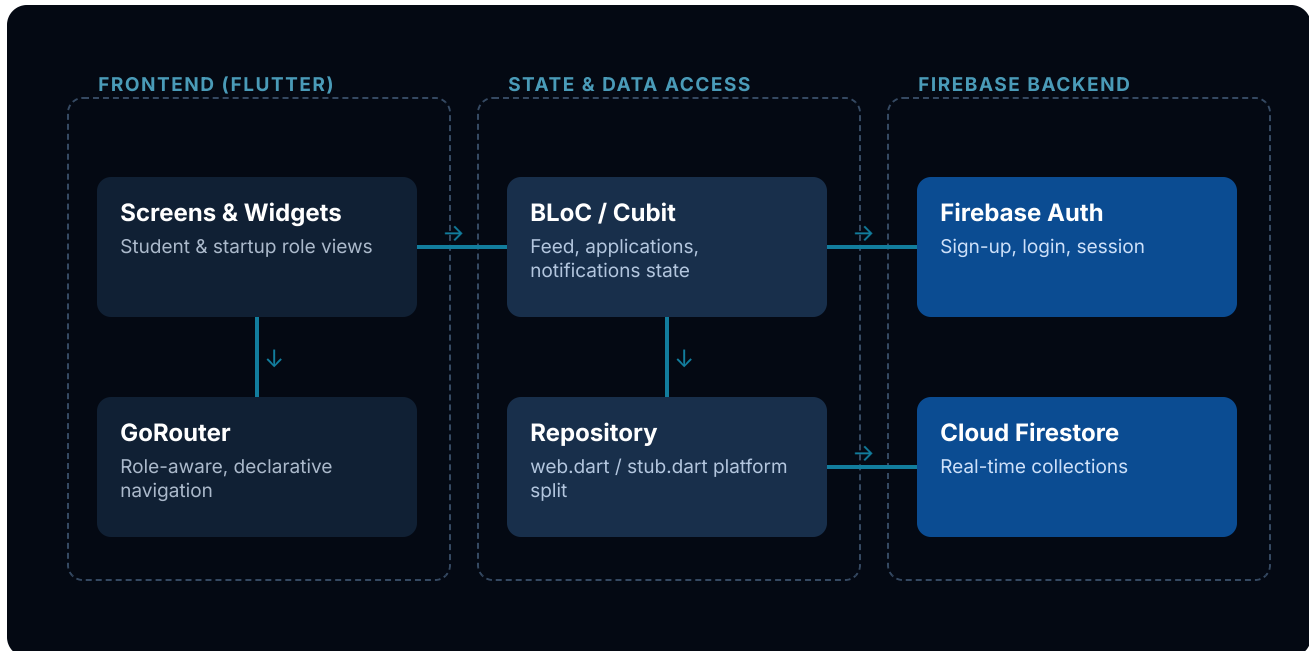


Figure 1. Layered application architecture. Screens dispatch events to BLoC/Cubit, which calls the Repository — the sole layer that talks to Firestore, and the one place the small amount of platform-specific code (conditional `web.dart` / `stub.dart` imports) lives. GoRouter drives role-aware navigation; Firebase Auth and Firestore stream state back up in real time.

- **UI layer** — stateless widgets that render based on BLoC/Cubit state and dispatch user events upward; contains no direct Firestore calls.
- **BLoC/Cubit layer** — owns feature state (auth, opportunity feed, applications, notifications) and mediates between UI events and repository calls.
- **Repository layer** — the only layer that talks to Firestore directly, so backend calls can be mocked in tests and swapped without touching UI or state code.
- **GoRouter** — declarative, role-aware routing (e.g., redirecting unverified startups to a pending-verification screen).
- **Conditional imports** — a `web.dart` / `stub.dart` pair abstracts any platform-specific API (see Section 7) so the same feature code compiles for both targets.

3. Firebase Backend Structure & Database Schema

ALU Connect uses Cloud Firestore as its sole backend, structured as six top-level collections. Relationships are modeled through document ID references rather than subcollections, which keeps queries simple and avoids deep nesting as the dataset grows.

Collection	Key Fields	References
users	uid, fullName, role, skills[], program	—
startups	id, name, isVerified	ownerId → users
opportunities	id, title, category, status (open/closed)	startupId → startups
applications	id, status (pipeline), isStarred	applicantId → users, opportunityId → opportunities
notifications	id, isRead	userId → users, relatedId → applications
bookmarks	id	userId → users, opportunityId → opportunities

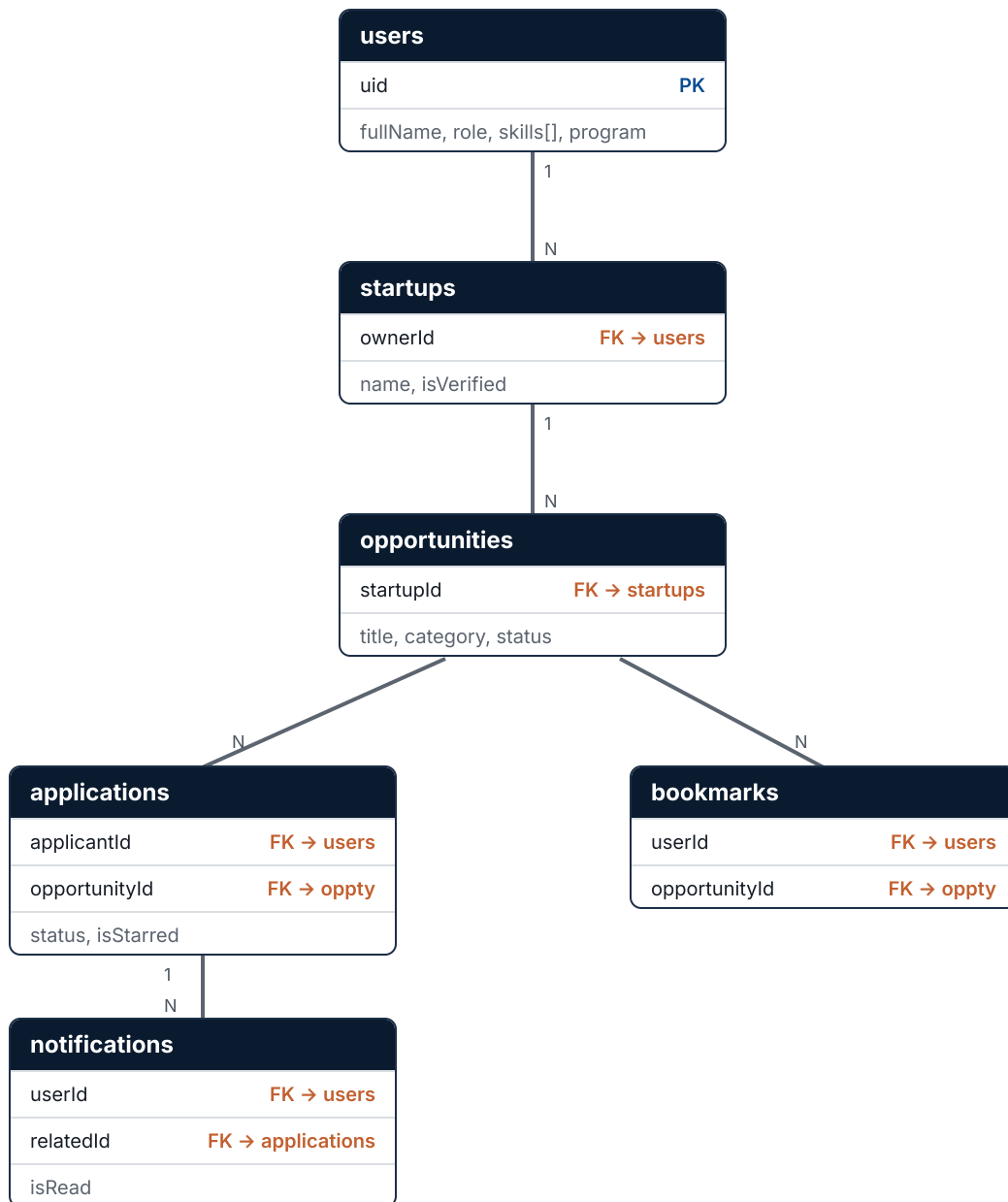


Figure 2. Entity relationship diagram. Lines trace the primary one-to-many chain (users → startups → opportunities → applications/bookmarks → notifications, labeled 1/N); each card's FK rows list its remaining references, including the two collections (applications, bookmarks) that also key off users directly.

4. State Management Approach

State is managed with the BLoC/Cubit pattern from `flutter_bloc` [4]. Each feature area (authentication, opportunity feed, application pipeline, notifications) owns a Cubit that exposes immutable state classes; widgets subscribe via `BlocBuilder/BlocListener` and rebuild only

when relevant state changes. This keeps business logic out of widgets, makes each Cubit independently testable, and gives a single, traceable path from a Firestore write to a UI update.

5. Application Workflows & CRUD Integration

The four Firestore CRUD operations map directly onto the platform's core workflows:

Operation	Workflow
Create	Student submits an application → a document is written to <code>applications</code> . Startup posts a new opportunity → a document is written to <code>opportunities</code> .
Read	Real-time Firestore snapshot listeners stream <code>opportunities</code> and <code>applications</code> data directly into the UI via <code>BlocBuilder</code> , so lists update live without manual refresh.
Update	A startup changes an application's status, which updates the <code>applications</code> document and writes a corresponding <code>notifications</code> document in the same operation, so the applicant is notified immediately.
Delete	A startup permanently deletes an opportunity, removing its document from <code>opportunities</code> entirely — distinct from closing an opportunity, which only updates its status field (see Section 7).

All four operations were verified live against the Firebase Console during development and demo, confirming that application-level actions correctly and immediately reflect in the backend.

6. UI/UX Reasoning & Key Features

The interface is organized around two distinct roles rather than a single generic feed, since students and startups need fundamentally different affordances.

Student	Startup
Role-based authentication	Registration with a verification gate
Opportunity discovery, search & category filter	Post, edit, close, and delete opportunities
Bookmarking opportunities	Applicants tab grouped by opportunity
Application submission with resume URL	Sort, filter, and star applicants
Application status tracking	Status changes trigger notifications
In-app notifications with bell badge	—

The startup verification gate was a deliberate design decision: without it, any user could pose as a startup and post fraudulent opportunities. Requiring verification before a startup can publish protects student trust in the platform, at the cost of an extra onboarding step for legitimate startups — a tradeoff judged worthwhile given the target audience.

7. Challenges Encountered & Lessons Learned

Challenge	Solution	Lesson
Firebase Storage is unavailable on the free (Spark) plan	Switched to a resume URL input, letting students link a Google Drive or Dropbox file instead of uploading directly	Design around platform and billing constraints early, before committing to a feature's implementation
<code>dart:html</code> crashes on non-web build targets	Adopted the conditional imports pattern (<code>web.dart / stub.dart</code>) to isolate platform-specific code	Flutter's write-once-run-anywhere promise still needs explicit platform abstraction for browser-only APIs
A Firestore composite-index requirement silently returned empty streams instead of erroring visibly	Replaced compound queries with single-field queries plus in-memory sorting	Always test real-time streams from a cold cache, not just a warm one where indexes may already exist
Users confused "close" and "delete" for opportunities — closed items appeared to vanish	Startups now see all opportunities including a "Closed" badge; Close (a status update) was strictly separated from Delete (a document removal)	A UI label and its underlying data behavior must match exactly, or users will misinterpret the system's state

8. Testing Strategy

Firestore CRUD paths were validated end-to-end by cross-checking every write, read, update, and delete against the Firebase Console in real time during development, ensuring application state and backend state never diverged. Cubit state transitions were exercised manually across cold-cache and warm-cache conditions after the composite-index issue in Section 7 surfaced the importance of testing both. Role-based navigation guards (e.g., blocking unverified startups from posting) were tested by exercising each role's flow independently.

9. Scalability Considerations

The repository layer's isolation of all Firestore access means the backend could later be swapped or fronted by Cloud Functions without touching UI or state code. Firestore's single-field query approach (adopted in Section 7) avoids composite-index maintenance overhead as

new filter combinations are added. The current lack of pagination (Section 10) is the main scalability gap identified for the present dataset size.

10. Limitations

- No push notifications — notifications are visible only within the app.
- No messaging or chat between students and startups.
- No pagination, which will affect performance as the opportunity and application datasets grow.

11. Future Improvements

- Firebase Cloud Messaging for real push notifications.
- In-app student–startup chat.
- A recommendation engine that matches opportunities to student skills.
- An admin verification dashboard for reviewing startup applications.

12. Screenshots of Key Workflows

Representative screenshots of the core workflows described above, captured from the running application and grouped by flow.

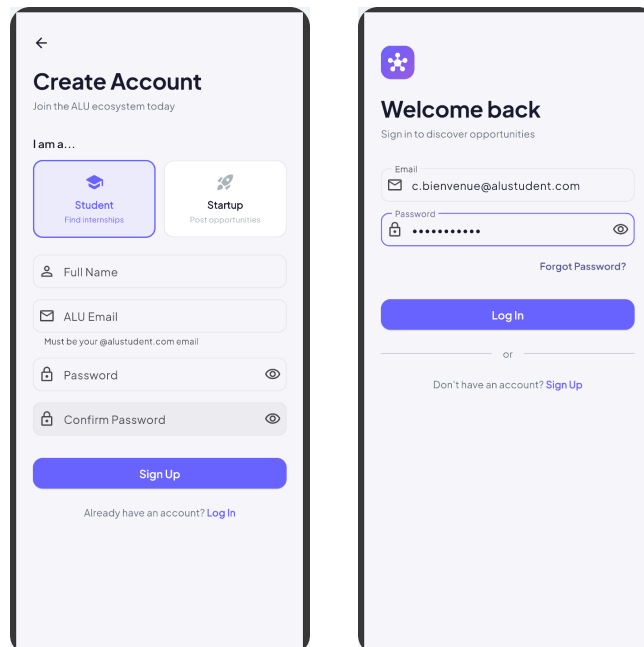


Figure 3. Authentication: role selection at sign-up, and returning-user login.

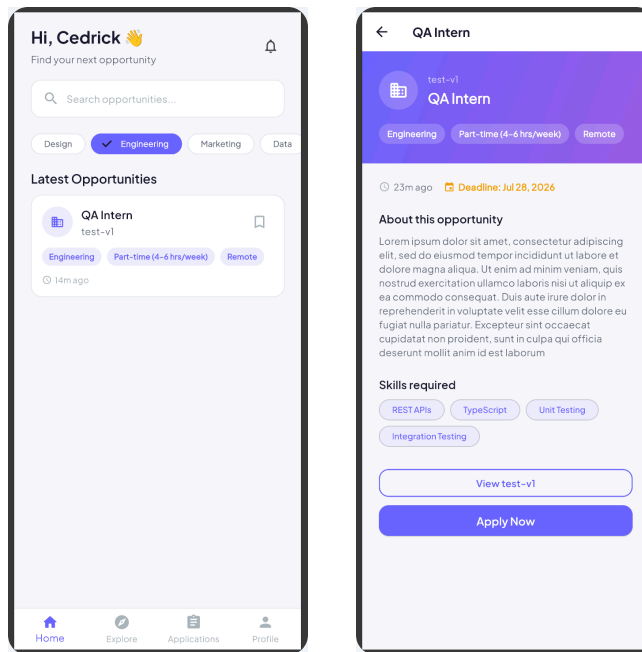


Figure 4. Opportunity discovery: category-filtered feed, then a single opportunity's detail view.

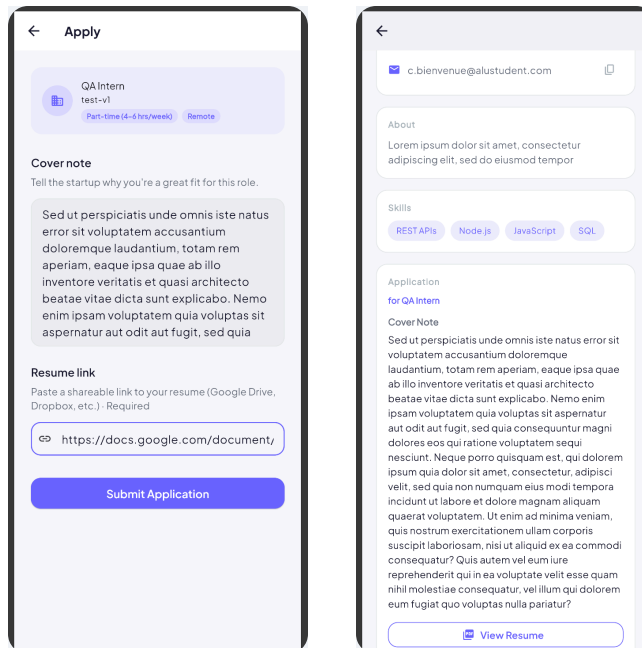


Figure 5. Application flow: submitting a cover note and resume link, then tracking status from the student profile.

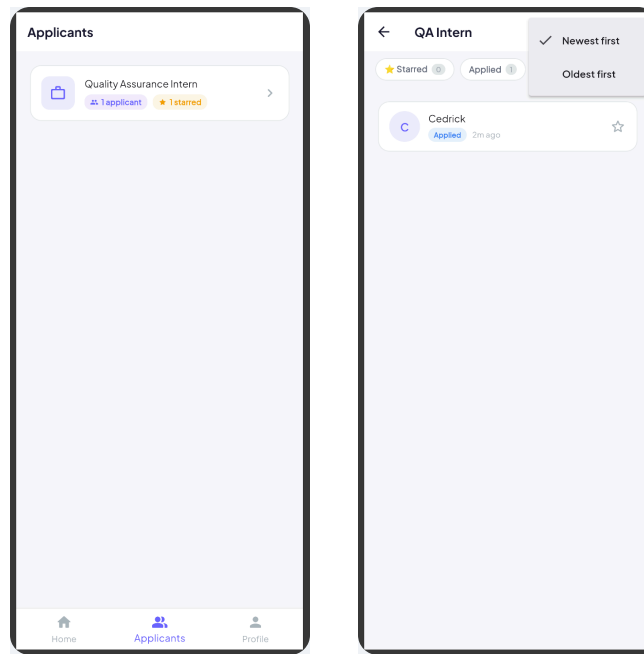


Figure 6. Startup side: applicants grouped by opportunity, then sorting and starring within one opportunity.

References

1. Flutter Team, "Flutter documentation," Google. [Online]. Available: <https://docs.flutter.dev>. [Accessed: Jul. 12, 2026].
2. Google, "Cloud Firestore for Flutter," Firebase Documentation. [Online]. Available: <https://firebase.google.com/docs/firestore>. [Accessed: Jul. 12, 2026].
3. Google, "Get started with Firebase Authentication for Flutter," Firebase Documentation. [Online]. Available: <https://firebase.google.com/docs/auth/flutter/start>. [Accessed: Jul. 12, 2026].
4. F. Angelov, "flutter_bloc," Bloc Library. [Online]. Available: <https://bloclibrary.dev>. [Accessed: Jul. 12, 2026].
5. "go_router package," pub.dev. [Online]. Available: https://pub.dev/packages/go_router. [Accessed: Jul. 12, 2026].